

图的表示与遍历

关系怎样被表示，并从起点覆盖而不重复失控

408 · 数据结构 Topic DS-07

母问题：图的关系如何变成可验证的访问过程？

图遍历的生成链是

Vertices/Edges → Representation → Frontier → Visited State → Expansion → Coverage.

邻接矩阵用空间换常数时间的边查询，邻接表按实际边数保存关系；DFS 与 BFS 都要维护 visited，区别在 frontier 的 LIFO/FIFO 取出规则。遍历只负责覆盖和发现关系，不自动解决最短路、MST 或拓扑目标。

本册定位与 Owner 边界

- **Owns**：图、邻接矩阵/邻接表/十字链表/邻接多重表、方向与重边语义、DFS/BFS、visited、连通分量与遍历森林。
- **Uses**：DS02 的栈/队列端点契约和 DS-B01 frontier 接口；DS08 使用本册表示。
- **Stops**：最短路、最小生成树、拓扑排序等全局目标归 DS08；网络路由只 Use。

目录

1	表示先于算法	3
1.1	十字链表：一条有向弧同时进入出边链与入边链	3
1.2	邻接多重表：无向边是共享对象，而不是两份边副本	3
1.3	邻接矩阵还可以承载路径计数	3
2	visited 是覆盖不变量	3
3	DFS、BFS 与遍历森林	4
3.1	四个动词固定遍历状态	4
3.2	图的基本计数：先固定对象，再套公式	4
3.3	BFS 的距离、前驱与路径恢复	5
3.4	遍历实现的等价改写与边界	5
3.5	DFS 的颜色状态与有向环	5
4	隐式状态图：先定义邻接，再选择 BFS 或 DFS	6
4.1	双向 BFS 的成立条件	6
4.2	Flood Fill 与岛屿族：变体来自“统计什么”	6
5	代码与测试证据	6

6	边界与概念分离	8
7	做题控制协议	9

1 表示先于算法

邻接矩阵 $A[i][j]$ 直接记录边存在性或权值，适合稠密图和频繁边查询，空间 $\Theta(V^2)$ 。邻接表为每个顶点保存出边列表，空间 $\Theta(V + E)$ ，遍历一个顶点的邻居只扫描实际出边。无向边需要在两端各登记一次；有向边只登记出发端。平行边和自环是否允许，必须在接口中先固定。

1.1 十字链表：一条有向弧同时进入出边链与入边链

普通有向邻接表让枚举顶点 u 的出边很自然，但若枚举入边，往往还需逆邻接表。十字链表把一条弧记录为

$$(tail, head, tlink, hlink, info),$$

其中 $tlink$ 连接同弧尾的下一条出边， $hlink$ 连接同弧头的下一条入边；顶点记录同时保存首条出弧和首条入弧。一个边对象被两条链引用，但只存一份权值和附加信息。

不变量是：每条弧在其尾顶点的出边链中出现一次，也在其头顶点的入边链中出现一次；删除弧必须同时从两条链摘除。它适合同时需要入度、出度和双向邻接扫描的有向图。

1.2 邻接多重表：无向边是共享对象，而不是两份边副本

无向邻接表通常把边 $\{u, v\}$ 分别登记在 u, v 的链中。邻接多重表为每条无向边只建一个边节点，保存两个端点 $ivex, jvex$ 以及分别连接两个端点下一条关联边的 $ilink, jlink$ 。从顶点 u 扫描时，要根据 u 是该边的哪一个端点选择对应 $link$ 。

其收益是边级插删与标记只作用于一个共享对象；代价是遍历 $link$ 时必须判断当前端点角色。十字链表面向有向弧的入/出双链，邻接多重表面向无向边的两端关联，二者不能只凭“都有两条链”互换。

1.3 邻接矩阵还可以承载路径计数

无权图的邻接矩阵 A 中， $(A^k)_{ij}$ 等于从 i 到 j 的长度恰为 k 的游走条数；这里允许重复顶点或边，因而是 **walk count**，不等于简单路径数。矩阵乘法的中间求和正好枚举最后一步来自哪个中间顶点。这个结论适合小图或固定步数的计数题，复杂度由矩阵乘法决定，不能拿来替代一般 DFS/BFS 的可达性判定。

邻接矩阵读度时必须先固定方向和对角线语义：无向图一行（或一列）非零项数给度；有向图行和是出度、列和是入度；若有自环，度数是否按 2 计必须与题目约定一致。带权图用 ∞ 表示无边时，0 只能表示零权边或自身距离，不能把两者混为同一个哨兵。

表示选择的题面信号

只需判断任意两点是否相邻且图较稠密，优先邻接矩阵；按实际边枚举出边，邻接表足够；有向图同时频繁枚举入边与出边，看十字链表；无向图需要对同一边做删除、标记或边级维护，看邻接多重表。选择依据是需要沿哪个关系方向访问，而不是结构名称的新旧。

2 visited 是覆盖不变量

遍历的核心不变量是：每个顶点最多被“首次发现”一次，已发现但未展开的顶点位于 **frontier**，展开后其出边已按表示规则检查。非连通图从一个起点只能覆盖一个可达分量；要得到遍历森林，必须在所有顶点上补一层外循环。

同一关系，不同 frontier

从 0 出发，若先把邻居压栈，得到 DFS 的深路径；若把邻居入队，得到 BFS 的层次距离。两者都可以覆盖同一可达集，但只有无权图 BFS 的首次发现层数具有最短边数语义。

3 DFS、BFS 与遍历森林

递归 DFS 隐式使用调用栈；显式 DFS 必须约定邻接顺序，否则输出序列不唯一。BFS 使用队列，入队时就标记 visited，不能等出队才标记，否则多个前驱会重复入队。对邻接表，二者时间均为 $O(V + E)$ ，额外空间为 $O(V)$ ；矩阵扫描每个顶点所有候选邻居，时间为 $O(V^2)$ 。

3.1 四个动词固定遍历状态

把任何遍历逐行代码都翻译成四个动作：

$$Discover \rightarrow Mark \rightarrow Enqueue/ Push \rightarrow Expand$$

Discover 是第一次遇到未访问邻居；Mark 把“已发现”写入 visited；Enqueue/ Push 把它加入待处理边界；Expand 是将来取出后扫描邻接表。Mark 必须紧跟 Discover，因为 frontier 中的顶点已经被别的路径“预约”，不能再次加入。

为什么出队时标记会重复

设 0 同时连接 1, 2，且 1, 2 都连接 3。BFS 展开 1 时发现 3 并入队；若尚未标记，展开 2 时又会把 3 入队。发现时标记则把状态从 ‘Unseen’ 立即改为 ‘Discovered’，第二条边只被检查，不再制造第二份 frontier 项。

顶点状态	所在位置	已经保证什么	下一动作
Unseen	不在 frontier	尚无已知发现路径	被邻边 Discover
Discovered	frontier 中	已有一条发现路径，visited=true	等待取出
Expanded	frontier 外	所有邻边已扫描	不再展开

DFS 与 BFS 共享这三态；区别只在 Discovered 顶点按 LIFO 还是 FIFO 取出。遍历森林再增加外层动作：找最小的 Unseen 顶点作为新根，重复同一状态机。

3.2 图的基本计数：先固定对象，再套公式

无向简单图的边是无序顶点对，最多有 $\binom{V}{2}$ 条边；有向简单图的弧是有序顶点对，最多有 $V(V - 1)$ 条弧。无向图中自环是否允许会改变度数口径：一条普通边对两个端点各贡献 1，自环对同一顶点贡献 2。因此在无自环无向图中

$$\sum_{v \in V} \deg(v) = 2E,$$

有向图则分别满足 $\sum \text{indeg}(v) = E$ 与 $\sum \text{outdeg}(v) = E$ 。

连通性判断必须和边数的前提一起读。无向图若 $E < V - 1$ 必不连通；若 $E > V - 1$ 必有环； $E = V - 1$ 既不能单独推出连通，也不能单独推出无环。连通图的生成树恰有 $V - 1$ 条边，非连通图的遍历结果是生成森林。对简单无向图，若

$$E > \frac{(V - 1)(V - 2)}{2},$$

则必连通；因为不连通时最多把 $V - 1$ 个顶点组成完全图、剩一个孤立点。严格大于不可改成大于等于，等号处存在不连通边界例子。

有向图还要区分弱连通、单向连通与强连通；把方向抹去只能证明基图连通，不能推出强连通。无向图可用 BFS/DFS 染色判断二部性：遇到已染色邻点同色即发现奇环，二部图等价于不含奇数长度回路。以上计数和染色都是遍历的直接扩展，不能把“访问过所有顶点”误写成“已经得到最短路”。

有向强连通简单图至少需要一个有向环覆盖所有顶点，因此 $E \geq V$ ；上界仍为 $V(V - 1)$ 。若有向边数满足 $E > (V - 1)(V - 2)$ ，只能推出把方向抹去后的基图连通，不能推出强连通：所有弧朝同一侧仍可能使任意两点无法互达。题目写“有向图连通”时先确认是弱连通、单向连通还是强连通。

由边数反推无向图分量数时，成功连接两个不同分量的边每条最多让分量数减一，所以 $c \geq \max(V - E, 1)$ ；这个下界可由一棵森林取到。若要让分量尽量多，应把边集中在最少的 k 个顶点中，其中 k 是满足 $\binom{k}{2} \geq E$ 的最小整数，其余顶点孤立，于是 $c \leq V - k + 1$ 。这是计数构造，不是遍历顺序结论。

3.3 BFS 的距离、前驱与路径恢复

对无权图，从源点 s 入队时置 $dist[s] = 0$ ，首次发现 v 时令

$$dist[v] = dist[u] + 1, \quad parent[v] = u.$$

队列的 FIFO 顺序保证顶点按非降距离层被展开；因此首次发现就是最少边数距离。若题目只问可达性，保留 visited 即可；若问具体路径，必须额外保存 parent，并从终点反向回溯后再逆序输出。不可达点保留 ∞ ，不能把“没有前驱”当成距离 0。

若每条边权都相同为常数 $c > 0$ ，BFS 仍给出最少边数路径，实际距离为 $c \cdot dist$ ；若边权不同，层号不再代表代价，应转交 Dijkstra 或其他最短路算法。若题目要求最短路径条数，首次发现时令 $count[v] = count[u]$ ；再次从同一最短层到达 v 时累加 $count[u]$ ，但只有满足 $dist[u] + 1 = dist[v]$ 的边才可计入。这个计数状态和 parent 选择是两个不同接口，不能用“最后一次到达的父节点”代替路径数。

3.4 遍历实现的等价改写与边界

递归 DFS 的调用栈记录“当前顶点及其下一条尚未检查的邻边”；显式栈实现若只压入顶点而不记录邻接游标，虽然仍可覆盖，但输出次序可能与递归版本不同。BFS 若用层级循环，可在每层开始记录当前队列长度；若只要距离，则直接传播 $dist$ 更不易漏层。空图、孤立顶点、重边、自环和非连通图必须分别测试：自环只应被 visited 拦截，重边不应生成重复发现，外层循环才负责补齐其他分量。

3.5 DFS 的颜色状态与有向环

在有向图中，用白色（未发现）、灰色（已发现但递归尚未返回）、黑色（全部邻边处理完）记录 DFS 状态。沿边遇到灰色顶点即得到后向边，说明当前递归栈形成有向环；遇到黑色顶点不能据此判环，因为它属于已经完成的另一分支。无向图若把父边排除后仍遇到已访问邻点，则得到环证据，但重边和自环必须单独处理。

DFS 生成树的先序取决于邻接访问顺序；因此题目若要求唯一序列，必须给出邻接表顺序或顶点扫描顺序。若只问是否存在环、连通分量数或覆盖集合，序列不唯一不影响结论。把 DFS 的“访问顺序”直接当作拓扑序是错误的，拓扑目标应交给 DS08 的逆后序/Kahn 不变量。

4 隐式状态图：先定义邻接，再选择 BFS 或 DFS

网格、密码锁、字符串变换等题目没有显式邻接表，但只要能定义“一个合法操作把状态 x 变成哪些状态”，就已经得到图。建模顺序固定为

State $\rightarrow$ Neighbor Generator $\rightarrow$ Forbidden/Visited $\rightarrow$ Goal $\rightarrow$ Frontier.

状态编码必须保留未来转移所需信息；visited 应在入队/进入递归时标记，而不是出队后才标记，否则同一状态可能被多个前驱重复加入。要求最少操作次数且每步代价相同时使用 BFS；只问覆盖、分量或自底向上汇总时通常使用 DFS。状态总数有限但无法一次列出时，复杂度写成 $O(|S| + |T|)$ ，其中 S 是实际发现状态、 T 是实际生成转移，不能脱离状态编码只写“ $O(n)$ ”。

4.1 双向 BFS 的成立条件

已知单一起点与终点、边可逆或能够生成反向邻居时，可从两端同时维护 frontier，每轮扩展节点数更少的一侧；新状态若已经在另一侧 visited 集合中，路径在此相接。它减少的是常见搜索深度下的状态展开量，不改变最坏状态空间阶。若终点未知、有向边没有可构造的反向操作，或两侧使用了不一致的状态规范化，双向 BFS 的相遇证明不成立。

4.2 Flood Fill 与岛屿族：变体来自“统计什么”

二维网格把每个合法单元格视为顶点，四连通或八连通决定邻接。Flood Fill 从种子出发覆盖同色连通分量；若新颜色等于旧颜色，应立即返回，避免反复生成等价转移。visited 可用独立布尔表，也可原地改写网格，但后者会破坏输入，必须写入 API 合同。无论 DFS 还是 BFS，最坏都可能保存 $O(mn)$ 个待处理单元，不能只凭“是岛屿题”宣称队列空间为 $O(\min(m, n))$ 。

岛屿计数在每次遇到未访问陆地时启动一次遍历；最大面积让一次遍历返回节点数；封闭区域或飞地可先从边界陆地遍历并消去，再统计内部剩余；子岛屿需在覆盖第二张图的分量时累计“所有对应位置在第一张图均为陆地”的合取条件。不同岛屿形状可记录 DFS 方向序列，但必须加入回退记号，使不同分支结构不会编码成同一串；这种相对位移编码通常只消除平移，不会自动消除旋转或镜像，若题目把它们视为同形，还需额外规范化。

网格题的停止条件

先固定越界、阻塞、已访问三类拒绝条件，再定义一次访问产生什么不可逆进展。若遍历会修改原网格，调用结束后是否需要恢复必须提前决定；回溯需要撤销“路径选择”，普通连通覆盖通常不撤销 visited。

5 代码与测试证据

完整实现位于 code/ds07_graph_traversal.hpp，测试位于 code/ds07_graph_traversal_test.cpp。

```
1 class Graph {
2     public:
3         explicit Graph(std::size_t vertices, bool directed = false)
4             : adjacency_(vertices), matrix_(vertices, std::vector<bool>(vertices,
5                 false)), directed_(directed) {}
6         std::size_t size() const { return adjacency_.size(); }
7         void add_edge(std::size_t from, std::size_t to) {
```

```

7     check(from); check(to);
8     adjacency_[from].push_back(to);
9     matrix_[from][to] = true;
10    if (!directed_) adjacency_[to].push_back(from);
11    if (!directed_) matrix_[to][from] = true;
12 }
13 const std::vector<std::size_t>& neighbors(std::size_t vertex) const {
14     check(vertex); return adjacency_[vertex];
15 }
16 bool has_edge(std::size_t from, std::size_t to) const {
17     check(from); check(to); return matrix_[from][to];
18 }
19 std::vector<std::size_t> bfs(std::size_t start) const {
20     check(start); std::vector<bool> seen(size()); std::vector<std::size_t>
    order;
21     std::queue<std::size_t> frontier; frontier.push(start); seen[start] = true;
22     while (!frontier.empty()) {
23         auto vertex = frontier.front(); frontier.pop(); order.push_back(vertex);
24         for (auto next : adjacency_[vertex]) if (!seen[next]) { seen[next] = true
    ; frontier.push(next); }
25     }
26     return order;
27 }
28 std::vector<std::size_t> dfs(std::size_t start) const {
29     check(start); std::vector<bool> seen(size()); std::vector<std::size_t>
    order;
30     std::stack<std::size_t> frontier; frontier.push(start); seen[start] = true;
31     while (!frontier.empty()) {
32         auto vertex = frontier.top(); frontier.pop();
33         order.push_back(vertex);
34         for (auto it = adjacency_[vertex].rbegin(); it != adjacency_[vertex].rend
    (); ++it) {
35             if (!seen[*it]) {
36                 seen[*it] = true;
37                 frontier.push(*it);
38             }
39         }
40     }
41     return order;
42 }
43
44 private:
45     std::vector<std::vector<std::size_t>> adjacency_;
46     std::vector<std::vector<bool>> matrix_;
47     bool directed_;
48     void check(std::size_t vertex) const {
49         if (vertex >= size()) throw std::out_of_range("graph vertex");
50     }
51 };
52
53 } // namespace ipara::ds07
54

```

```
55 #endif
```

代码清单 1: 邻接表、BFS、DFS 与可达覆盖

```
1
2 int main() {
3     Graph graph(6);
4     graph.add_edge(0, 1); graph.add_edge(0, 2); graph.add_edge(1, 3);
5     graph.add_edge(2, 4);
6     assert(graph.has_edge(0, 1) && graph.has_edge(1, 0));
7     assert(!graph.has_edge(0, 5));
8     assert((graph.bfs(0) == std::vector<std::size_t>{0, 1, 2, 3, 4}));
9     assert((graph.dfs(0) == std::vector<std::size_t>{0, 1, 3, 2, 4}));
10    assert(graph.bfs(5).size() == 1);
11
12    Graph diamond(4, true);
13    diamond.add_edge(0, 1); diamond.add_edge(0, 2);
14    diamond.add_edge(1, 3); diamond.add_edge(2, 3);
15    assert((diamond.bfs(0) == std::vector<std::size_t>{0, 1, 2, 3}));
16    assert((diamond.dfs(0) == std::vector<std::size_t>{0, 1, 3, 2}));
17
18    Graph directed(2, true); directed.add_edge(0, 1);
19    assert(directed.neighbors(1).empty());
20    assert(directed.has_edge(0, 1) && !directed.has_edge(1, 0));
21    bool threw = false; try { graph.add_edge(6, 0); } catch (const std::
        out_of_range&) { threw = true; }
22    assert(threw);
23 }
```

代码清单 2: 方向、非连通与空图测试

```
1 clang++ -std=c++17 -Wall -Wextra -Werror -pedantic code/
    ds07_graph_traversal_test.cpp -o /tmp/ds07_test
2 /tmp/ds07_test
```

6 边界与概念分离

操作	邻接表	邻接矩阵	不变量/前提	边界
遍历	$O(V + E)$	$O(V^2)$	每顶点首次发现一次	空图、非连通
边查询	$O(\deg u)$	$O(1)$	方向语义固定	重边/自环
空间	$O(V + E)$	$O(V^2)$	只保存约定关系	稠密/稀疏

概念 A	≠ 概念 B	判据	错因
DFS	≠ BFS	栈 vs 队列	误写距离结论
首次发现	≠ 出队/出栈	visited 时机	重复入 frontier
遍历覆盖	≠ 最短路/MST	目标不同	把结构事实当目标证明

7 做题控制协议

先标顶点、方向、重边和表示；再确定起点与邻接访问顺序。看到“访问所有可达顶点”选 DFS/BFS，看到“最少边数”才升级到 BFS 距离，看到权值优化转交 DS08。每一步写出 visited 与 frontier 的变化，最后检查是否遗漏非连通分量。

压缩复原

五问：边存在哪里？当前 frontier 是栈还是队列？何时标记 visited？起点能覆盖哪些分量？题目要覆盖还是要优化？